Project Report on

Smart Logistics Optimization

Submitted to

VISHWAKARMA UNIVERSITY, PUNE

Under the Initiative of

Contemporary Curriculum, Pedagogy, and Practice

(C2P2)

Design and Analysis of

Algorithms

Made By :

C-62 Abdul Saied Maniyar

C-38 Ammar Shaikh

C-28 Afdal Nayab

Faculty In-charge: Prof. Alfiya Shahbad

Department of Artificial Intelligence

Faculty of Science & Technology

Academic Year

2026-2027 Term-I

Abstract

Logistics operations depend on connecting a warehouse with regional hubs and delivery points at the lowest possible distance or cost. Planning such connections and routes by hand is slow and can overlook cheaper alternatives. **RouteWise** is a web-based educational application that applies three classical graph algorithms to this problem. The road network is modelled as a weighted, undirected NetworkX graph of seven locations (Warehouse, North Hub, East Hub, South Hub, West Hub, Airport Depot and City Center) joined by thirteen road connections whose weights are illustrative distance units.

The application is written in Python with the Flask framework and NetworkX; its single template (`templates/index.html`) uses HTML, CSS and JavaScript. Through the routes `/mst`, `/shortest` and `/tsp` it computes a **Minimum Spanning Tree** with Kruskal's algorithm, the **shortest path** between two selected locations with Dijkstra's algorithm, and an exact **Travelling Salesman Problem (TSP)** tour by permutation search over all-pairs shortest distances. Results are shown as a result panel, a table of roads and a drawn network in which the selected roads are highlighted.

On the sample network, the total road weight is 44 units. The MST has 6 edges with a total weight of 14 units. The shortest path from Warehouse to Airport Depot is Warehouse → City Center → East Hub → Airport Depot, at 7 units. The TSP search evaluates 720 visit orders when the start is fixed, and for the default start (Warehouse) the best tour found has a length of 23 units. These values were confirmed by running the project code. The distances are sample values and do not represent live traffic or real map data.

Keywords: Minimum Spanning Tree, Kruskal's algorithm, Dijkstra's algorithm, Travelling Salesman Problem, graph algorithms, NetworkX, Flask, logistics route optimization.

Table of Contents

1. Introduction
2. Problem Statement
3. Aim and Objectives
4. Scope and Limitations
5. Background Concepts
6. System Requirements
7. System Architecture
8. Algorithm and Methodology
9. Implementation Details
10. Testing and Results
11. Complexity Analysis
12. Advantages and Applications
13. Future Enhancements
14. Conclusion

References

Appendix A: Application Screenshots

1. Introduction

Logistics is the planning and movement of goods from a source to their destinations. Every delivery operation, from a local courier to a national distribution network, depends on how well roads and routes are chosen. Warehouses must be linked efficiently to regional hubs and delivery points, because even a small saving on one route is repeated across many vehicles and many days.

A road network maps naturally onto a graph: locations become vertices and roads become weighted edges. Once the network is a graph, well-studied algorithms with proven correctness and known running times can be applied directly. Three practical questions arise repeatedly in logistics, and each corresponds to a classical graph problem:

Logistics question	Graph problem	Algorithm in RouteWise
Connect all locations using the least total road length	Minimum Spanning Tree (MST)	Kruskal's algorithm
Find the cheapest route between two locations	Shortest path	Dijkstra's algorithm
Visit every location and return to the start	Travelling Salesman Problem (TSP)	Exhaustive permutation search

RouteWise puts these three algorithms into a web application built with Flask and NetworkX. It also makes several Design and Analysis of Algorithms topics easy to see in practice: greedy choice, priority queues, shortest-path relaxation, factorial growth of exhaustive search and complexity analysis.

2. Problem Statement

Given a logistics network consisting of a warehouse and several delivery locations connected by roads with known distance values, the aim is to (i) find the minimum-cost set of roads that connects all locations, (ii) find the shortest route between two user-selected locations, and (iii) find the cheapest round trip that visits every location from a user-selected starting point. The results must be presented in a web interface so that they can be understood without reading code.

Manual planning has several weaknesses. People tend to choose familiar or visually short routes that are not actually the cheapest. The number of possible visiting orders grows factorially, so comparing them by hand quickly becomes impossible. Different planners may choose different routes for the same network, with no proof of optimality, and every change in a road cost forces the plan to be worked out again. RouteWise therefore uses established graph algorithms to obtain repeatable and provably correct results.

3. Aim and Objectives

Aim: To build a web-based logistics optimization application that demonstrates how a minimum spanning tree, shortest-path analysis and the Travelling Salesman Problem can be applied to a warehouse-and-hubs road network.

- Represent logistics locations as vertices and roads as weighted edges of an undirected NetworkX graph.
- Identify a minimum spanning tree of the network using Kruskal's algorithm.
- Calculate the shortest path between a selected source and destination using Dijkstra's algorithm.
- Find the best round-trip tour from a selected start location through all locations using exact permutation search.
- Present the result, a table of roads and a drawing of the network, with the selected roads highlighted, in the web interface.
- Display the time complexity of each algorithm and analyse its limits.
- Verify the results against the project code and explain the differences between an MST, a shortest path and a TSP tour.

4. Scope and Limitations

4.1 Scope

- One fixed sample network of seven locations and thirteen roads, defined in `app.py`.
- Kruskal's MST on the whole network (`/mst`).
- Dijkstra's shortest path between any two selected locations, chosen from drop-down lists (`/shortest`).
- An exact TSP tour from any selected start location (`/tsp`).
- An overview page with location count, road count, total road distance and a drawing of the network (`/`).
- Educational demonstration of algorithm design and analysis; the interface itself states that distances are sample values.

4.2 Limitations

- The network is a small sample with illustrative distance units. It is not live map, GPS or traffic data.
- The graph is defined in the source code. The interface does not let the user add or edit locations or roads.
- The TSP uses exhaustive search over $(V - 1)!$ orders. This is practical for seven locations but not for large networks.
- Only distance is minimised. Vehicle capacity, delivery time windows, multiple vehicles and road restrictions are not modelled.
- Edge weights must be non-negative for Dijkstra's algorithm. The sample data satisfy this.
- The application is run with the Flask development server (`debug=True`) and is intended for local demonstration, not production deployment.

5. Background Concepts

5.1 Weighted Undirected Graph Model

The network is a **weighted, undirected graph** $G = (V, E)$, created in the code with `nx.Graph()`. Each **vertex** is a location, each **edge** is a road between two locations and each **edge weight** is the distance of that road in illustrative units. The graph is undirected because a road can be travelled in both directions at the same cost. Here $|V| = 7$ and $|E| = 13$. Every location also stores screen coordinates (`pos`) that are used only to draw the network in the browser; the algorithms use the weights, not the coordinates.

5.2 Dataset: Locations and Road Connections

The seven locations are Warehouse, North Hub, East Hub, South Hub, West Hub, Airport Depot and City Center. The 13 roads below are taken directly from the `ROADS` list in `app.py`. Their weights add up to **44 units**, the value shown as "Total Road Distance" in the interface. This total is only a property of the dataset; it is not the weight of the MST or of a tour.

No.	From	To	Weight	No.	From	To	Weight
1	Warehouse	North Hub	4	8	City Center	East Hub	3
2	Warehouse	West Hub	3	9	City Center	South Hub	4
3	Warehouse	City Center	2	10	East Hub	Airport Depot	2
4	North Hub	West Hub	2	11	East Hub	South Hub	3
5	North Hub	City Center	3	12	South Hub	Airport Depot	5
6	North Hub	East Hub	5	13	West Hub	South Hub	6
7	West Hub	City Center	2				

A spanning tree on 7 vertices always has exactly 6 edges, and a TSP tour with a fixed start has 6! = 720 possible visit orders. The graph is connected, which both of these require.

5.3 Minimum Spanning Tree and Kruskal's Algorithm

A spanning tree connects all vertices without forming a cycle. A **minimum spanning tree** is the spanning tree whose total edge weight is smallest. Kruskal's algorithm builds it greedily: it sorts the edges by weight and adds the next edge only if it does not join two vertices that are already connected. An MST is a network-design result. It minimises the total weight used to connect everything, but it does not give the shortest route between two particular locations and it is not a delivery tour.

5.4 Shortest Path and Dijkstra's Algorithm

The shortest path between a source and a destination is the route with the smallest total weight. Dijkstra's algorithm finds it for graphs whose edge weights are non-negative. It keeps a tentative distance for every vertex, repeatedly settles the unvisited vertex with the smallest distance and relaxes the edges leaving it.

5.5 Travelling Salesman Problem

The Travelling Salesman Problem asks for the cheapest tour that visits every location and returns to the starting location. It is NP-hard, so no known algorithm solves every instance quickly. For a small network such as RouteWise, an exact answer can be found by trying every possible visiting order.

5.6 Related Algorithms

Algorithm	Core idea	Typical use
Prim's algorithm	Grows one tree by repeatedly adding the cheapest edge leaving it	Alternative MST method
Bellman–Ford	Relaxes all edges V − 1 times	Shortest paths with negative weights
Floyd–Warshall	Dynamic programming over intermediate vertices	All-pairs shortest paths
Held–Karp	Dynamic programming over subsets of locations	Exact TSP, $O(n^2\,2^n)$
Nearest neighbour / 2-opt	Greedy construction followed by local improvement	Heuristic TSP for larger networks

6. System Requirements

Component	Requirement / purpose
Hardware (suggested)	Dual-core processor, 4 GB RAM and less than 500 MB of free disk space; the application is lightweight
Operating system	Windows (project folder `C:\DAA`); Linux and macOS also work with Python and Flask
Python 3	Backend, graph construction and the TSP search (standard-library `itertools`)
Flask >= 3.0	Web framework: routes, request values and template rendering
NetworkX >= 3.3	Graph structure, Kruskal MST, Dijkstra and all-pairs shortest paths
Web browser	Any modern browser to display the interface and the SVG network drawing

Functional requirements. The system shall (1) build a weighted undirected graph of 7 locations and 13 roads and show the location count, road count and total road distance on every page; (2) compute and show the MST edges and total weight; (3) let the user select a source and a destination and show the shortest route, distance and per-road table; (4) let the user select a start location and show the best tour, its length and a table of its legs; and (5) draw the network with MST roads in green or the calculated route in red.

Non-functional requirements. Usability: drop-down inputs and a sidebar menu keep the interface simple. **Correctness**: results must be correct for the given graph, and invalid selections fall back to default locations. **Maintainability**: one backend file and one template, with data, graph creation and algorithms in separate functions. **Portability**: the application runs locally on any system with Python, the two packages and a browser. **Performance**: results for seven locations appear immediately in the tests carried out for this report, but no formal benchmark was performed.

Installation and execution. Install the packages listed in the requirements file (`pip install "Flask>=3.0" "networkx>=3.3"`), run `python app.py` in the project folder, and open the address printed by Flask, normally `http://127.0.0.1:5000/`.

7. System Architecture

RouteWise is a server-rendered Flask application. The system follows this sequence:

1. The user opens a page or submits a drop-down form in the browser.
2. Flask receives the request and reads the submitted values (`source`, `destination` or `start`). A value that is not a valid location is replaced by a default.
3. The backend builds the NetworkX graph with `create_graph()`.
4. The route runs the required algorithm: Kruskal's MST, Dijkstra's shortest path or the TSP permutation search.
5. `render_page()` adds the location count, road count, total distance, result and network data to the template and renders `templates/index.html`.
6. The browser displays the result panel and tables, and JavaScript draws the network as an SVG image from data embedded in the page, highlighting the MST roads or the calculated route.

The browser does not call separate data APIs. Every result is a fully rendered page, and all algorithms run in Python on the server. JavaScript is used only to draw the network.

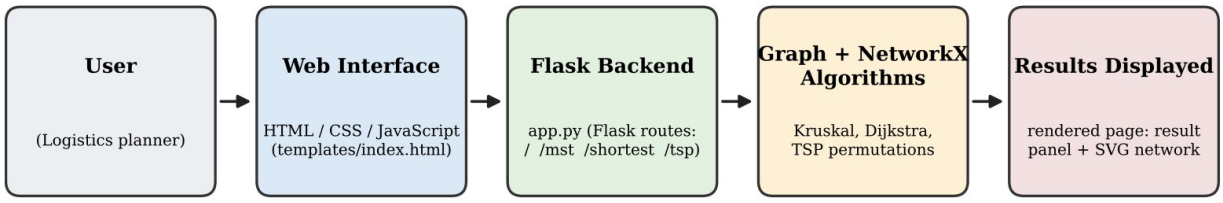


Figure 1. System architecture and data flow of RouteWise

8. Algorithm and Methodology

8.1 Minimum Spanning Tree: Kruskal's Algorithm

In RouteWise the MST is computed by the NetworkX built-in `nx.minimum_spanning_tree(graph, algorithm="kruskal", weight="weight")`, so the algorithm is not hand-written in the project. The steps of Kruskal's algorithm are:

1. Sort all edges by weight in ascending order.
2. Start with every vertex in its own set (Union-Find).

3. Take the next edge. If its two ends are in different sets, add the edge to the tree and merge the sets; otherwise skip it, because it would form a cycle.
4. Stop when the tree has V − 1 edges.

```
sort edges by weight (ascending); each vertex starts in its own set
for each edge (u, v, w) in sorted order:
    if find(u) != find(v):  add edge to MST; union(u, v)
    stop when MST has V - 1 edges
```

Time complexity: O(E log E), dominated by sorting the edges.

8.2 Shortest Path: Dijkstra's Algorithm

The route uses `nx.dijkstra_path` for the path and `nx.dijkstra_path_length` for its distance. The source and destination come from the two drop-down lists (defaults Warehouse and Airport Depot). The table of roads is rebuilt from consecutive vertices on the returned path. The method requires non-negative weights.

1. Set the distance of the source to 0 and all other vertices to infinity.
2. Repeatedly take the unvisited vertex with the smallest distance.
3. For each neighbour, relax the edge: if the distance through the current vertex is smaller, update it and record the predecessor.
4. When the destination is settled, rebuild the path from the predecessors.

```
dist[source] = 0; all others = infinity; push source into min-heap
while heap not empty:
    pop u with smallest dist
    for each neighbour v of u with weight w:
        if dist[u] + w < dist[v]: dist[v] = dist[u] + w; parent[v] = u
rebuild the path from destination back to source using parent[]
```

Time complexity (binary-heap implementation): O((V + E) log V).

8.3 Travelling Salesman Problem: Exact Permutation Search

The TSP route is written in the project itself. The tour is defined as starting at a chosen location, visiting every other location once and **returning to the start**, and the return leg is included in the cost. The method works in four stages:

1. Compute the shortest distance and the shortest road path between every pair of locations with `nx.all_pairs_dijkstra_path_length` and `nx.all_pairs_dijkstra_path`.
2. Fix the start location (default Warehouse) and generate every order of the remaining 6 locations with `itertools.permutations`: 6! = 720 candidate tours.
3. For each candidate tour = start + order + start, add the pairwise shortest distances of its legs. Keep the tour only if its cost is strictly smaller than the best so far, so the first of several equal tours is kept.
4. Expand each leg of the best tour into the physical roads it uses, using the stored shortest paths, and display the legs in a table.

```
D, Paths = all-pairs shortest distances and paths
for each order in permutations(all locations except start):
    tour = start + order + start
    cost = sum(D[tour[i]][tour[i+1]] for each leg)
    keep tour if cost < best_cost
expand each leg of best tour into roads using Paths
```

Time complexity: there are (V − 1)! candidate tours and costing each takes O(V), so the search is O(V·(V − 1)!) = O(V!). The interface displays O((V − 1)!), which counts the permutations. The all-pairs preprocessing adds V Dijkstra runs, which is negligible by comparison.

8.4 Simplified Request-Handling Logic

```
route /mst:
    mst = kruskal_mst(create_graph())
    show edges, total weight; highlight MST roads
route /shortest:
    source, destination = form values (default if invalid)
    path, distance = dijkstra(create_graph(), source, destination)
    show route, distance, road table; highlight path
route /tsp:
    start = form value (default if invalid)
    tour, cost = best_of_all_permutations(start)
    show tour, cost, leg table; highlight expanded path
```

9. Implementation Details

9.1 Backend: app.py

`app.py` imports Flask, NetworkX and `itertools`. It contains the data, the graph-building code, the helper functions and the four routes:

Function / component	Responsibility
`LOCATIONS`, `ROADS`	Dictionary of the 7 locations with drawing coordinates, and list of the 13 roads as (location, location, weight)
`create_graph()`	Builds the undirected `nx.Graph` with node positions and weighted edges
`get_total_distance()`	Sums the edge weights of a graph (44 for the full network, 14 for the MST)
`build_network_data()`	Converts the graph into nodes and edges for drawing and tags each edge as `normal`, `mst` or `path`
`render_page()`	Shared function that renders `index.html` with counts, total distance, result and network data
`dashboard()` /	Overview page with the network drawing and links to the algorithms
`minimum_spanning_tree()` /mst	Runs Kruskal's MST and highlights its roads
`shortest_path()` /shortest	Validates source and destination, runs Dijkstra, returns route, distance and road table
`travelling_salesman()` /tsp	Validates the start, runs the permutation search, expands the best tour and returns its legs

The `/shortest` and `/tsp` routes accept both GET and POST, so they can be opened directly or submitted from their forms. The application is started with `app.run(debug=True)`.

9.2 Frontend: templates/index.html

The single template contains a sidebar with four links (Overview, Minimum Spanning Tree, Shortest Path, Delivery Tour (TSP)); four metric cards (Total Locations, Road Connections, Total Road Distance, DAA Algorithms); a Road Network Visualization drawn as SVG with a legend; an Algorithm Result panel with the result value, route, complexity badge and table; a Road Connections table; and an Algorithm Reference card. The Shortest Path page shows two drop-downs (Starting Location and Destination) and a "Calculate Shortest Route" button. The TSP page shows one drop-down (labelled Starting Warehouse, although it lists all seven locations) and an "Optimize Delivery Tour" button. Both forms are submitted with POST. The CSS is embedded in the template and includes responsive rules for narrow screens. The JavaScript reads the network data embedded in the page and draws roads, weight labels and location nodes; it does not run the algorithms.

9.3 Application Workflow

1. Run `python app.py` and open the local address in a browser; the Overview page loads with the network drawing.
2. Choose Minimum Spanning Tree, Shortest Path or Delivery Tour (TSP) from the sidebar.
3. On the Shortest Path or TSP page, select the locations from the drop-down lists and submit the form.
4. Flask builds the graph, runs the algorithm and renders the result into the template.
5. The page shows the result, complexity badge and table, and the network drawing highlights the MST roads (green) or the calculated route (red).

10. Testing and Results

The values below were obtained by running the logic in the supplied `app.py` (NetworkX 3.6.1, Flask 3.1.3) and by opening its pages in a browser. They apply to this sample network only.

Measure	Result
Locations / road connections	7 / 13
Sum of all road weights	44 units
Kruskal MST	6 edges, total weight 14 units
Shortest path Warehouse → Airport Depot	Warehouse → City Center → East Hub → Airport Depot, 7 units (2 + 3 + 2)
TSP search with the start fixed	720 visit orders evaluated
Best tour from Warehouse (default start)	23 units; four visit orders tie at 23 (two cycles, each in both directions) and the first one found is shown

10.1 Minimum Spanning Tree Result

From	To	Weight
Warehouse	City Center	2
West Hub	City Center	2
North Hub	West Hub	2
East Hub	Airport Depot	2
East Hub	City Center	3
East Hub	South Hub	3
Total (6 edges)		**14**

The MST needs only 14 of the 44 available units and still connects every location. It tells the planner which roads are essential for connectivity, not how to travel between two locations.

10.2 Shortest Paths from the Warehouse

Destination	Shortest path	Distance
North Hub	Warehouse → North Hub	4
East Hub	Warehouse → City Center → East Hub	5
South Hub	Warehouse → City Center → South Hub	6
West Hub	Warehouse → West Hub	3
Airport Depot	Warehouse → City Center → East Hub → Airport Depot	7
City Center	Warehouse → City Center	2

The 7-unit path to Airport Depot uses three roads. It is a single route and must not be confused with the MST weight (14) or the tour length (23).

10.3 TSP Result for the Default Start (Warehouse)

From	To	Distance
Warehouse	West Hub	3
West Hub	North Hub	2
North Hub	East Hub	5
East Hub	Airport Depot	2
Airport Depot	South Hub	5
South Hub	City Center	4
City Center	Warehouse	2
Best tour length		**23**

Each leg of this tour is a single direct road, so expanding it into physical roads adds no intermediate stops. For the start South Hub the program also returns a tour of 23 units; there the leg Airport Depot → North Hub is a shortest path through East Hub (7 units) and is expanded into two roads. A tour is a cycle, so its length does not depend on which location is chosen as the start.

10.4 Test Cases

Test case	Expected result	Status
Graph creation	7 vertices, 13 edges, total weight 44	Verified by running `app.py` logic
MST calculation	6 edges, 14 units	Verified by running `app.py` logic
Shortest path Warehouse → Airport Depot	Path above, 7 units	Verified by running `app.py` logic
Shortest path, same source and destination	Single location, 0 units	Verified with the Flask test client
TSP, start Warehouse	720 orders evaluated, best length 23	Verified by running `app.py` logic
TSP, start South Hub	Best length 23	Verified with the Flask test client
Invalid source, destination or start	Defaults are used	Verified with the Flask test client
Pages `/`, `/mst`, `/shortest`, `/tsp`	Each page loads without error	Verified with the Flask test client
Network drawing in a browser	13 roads and 7 locations drawn; 6 MST roads (green), 3 path roads (red), 7 tour roads (red)	Verified in headless Chromium; screenshots in Appendix A

These checks were carried out once by scripts for this report. They are not a formal automated test suite.

10.5 Discussion

The three results answer three different questions on the same network. The MST (14 units) is the cheapest way to connect all locations, the shortest path (7 units) is the cheapest way to travel between two particular locations, and the tour (23 units) is the cheapest round trip through all locations. The tour is longer than the MST because it must return to the start and each location is entered and left. The exhaustive search guarantees that 23 is the minimum for this network because all 720 orders were evaluated.

11. Complexity Analysis

Stage	Time complexity	Explanation
Graph creation	$O(V + E)$	Adds 7 nodes and 13 edges
Kruskal MST	$O(E \log E)$	Sorting the edges dominates; Union-Find is nearly constant per edge
Dijkstra (one pair)	$O((V + E) \log V)$	Binary-heap implementation
All-pairs shortest paths	$O(V (V + E) \log V)$	Dijkstra run from each of the V locations
TSP permutation search	$O(V \cdot (V - 1)!) = O(V!)$	$(V - 1)!$ tours, each costed in $O(V)$; the interface shows $O((V - 1)!)$
Space	$O(V + E)$ plus $O(V^2)$	Graph plus the distance and path tables; `itertools.permutations` generates tours one at a time

Theoretical complexity describes how running time grows with input size. It is not a measured execution time. For the seven-location network the three algorithms run quickly, but the TSP search is the only stage whose cost grows explosively:

Number of locations (V)	Visit orders (V – 1)!
7 (this project)	720
8	5,040
10	362,880
12	39,916,800
15	87,178,291,200

This growth is why larger instances need dynamic programming (Held–Karp) or heuristics instead of exhaustive search.

12. Advantages and Applications

- Connects Design and Analysis of Algorithms theory with a working web application.
- Demonstrates a greedy algorithm (Kruskal), a priority-queue based algorithm (Dijkstra) and exhaustive search with factorial growth (TSP) on one dataset.
- Gives provably optimal results for the MST and the shortest path, and an optimal tour for the small network.
- Results are repeatable and appear quickly for the sample network.
- The network drawing with highlighted roads helps users see the difference between an MST, a path and a tour.
- The modular structure (data, graph creation, algorithms, template) makes it easy to change the dataset or add algorithms.
- The same ideas apply to network design (laying cables or pipelines), route finding in navigation and delivery planning, once real data are used.

13. Future Enhancements

The following items are proposals and are **not** part of the current implementation.

- Use real road-network data and map visualisation instead of sample weights and a drawn schematic.
- Add live traffic information and time-dependent edge costs.
- Allow users to add or edit locations and roads, and support larger graphs.
- Add Held–Karp dynamic programming or heuristic methods (nearest neighbour, 2-opt) for larger TSP instances.
- Model vehicle capacity, delivery time windows and several vehicles.
- Add input validation messages, automated tests and measured performance evaluation.
- Show the TSP tour order on the drawing and compare it with heuristic results.

14. Conclusion

RouteWise addresses the problem of planning efficient connections and routes in a logistics network. The network is modelled as a weighted undirected graph of seven locations and thirteen roads. Kruskal's algorithm shows how all locations can be connected at minimum total weight (6 edges, 14 units). Dijkstra's algorithm gives the cheapest route between two chosen locations (7 units from Warehouse to Airport Depot). The exact permutation-based TSP search evaluates 720 visit orders and, using all-pairs shortest distances, finds a best tour of 23 units from the default start, with its legs expanded into real roads. Combining the three techniques in a Flask and NetworkX application shows how algorithm design and analysis can support logistics decisions, and also shows why exhaustive TSP

search does not scale. The current implementation is an educational prototype that uses sample data, so no real-world savings or performance gains are claimed.

References

[1] Python Software Foundation, "Python Documentation." https://docs.python.org/3/

[2] Python Software Foundation, "itertools — Functions creating iterators for efficient looping." https://docs.python.org/3/library/itertools.html

[3] Pallets Projects, "Flask Documentation." https://flask.palletsprojects.com/

[4] NetworkX Developers, "NetworkX Documentation." https://networkx.org/documentation/stable/

[5] A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," Proceedings of the 7th Python in Science Conference (SciPy 2008), pp. 11–15, 2008.

[6] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," Proceedings of the American Mathematical Society, vol. 7, no. 1, pp. 48–50, 1956. https://doi.org/10.1090/S0002-9939-1956-0078686-7

[7] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269–271, 1959. https://doi.org/10.1007/BF01386390

[8] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," Journal of the Society for Industrial and Applied Mathematics, vol. 10, no. 1, pp. 196–210, 1962.

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

[10] D. L. Applegate, R. E. Bixby, V. Chvátal and W. J. Cook, The Traveling Salesman Problem: A Computational Study, Princeton University Press, 2006. https://press.princeton.edu/books/hardcover/9780691129938/the-traveling-salesman-problem

Appendix A: Application Screenshots

The following screenshots were taken from the running RouteWise application (Flask development server, opened in a browser). They show the pages and the results described in Section 10.

Figure A.1: Overview page showing the 7 locations, 13 roads and total road distance of 44 units.

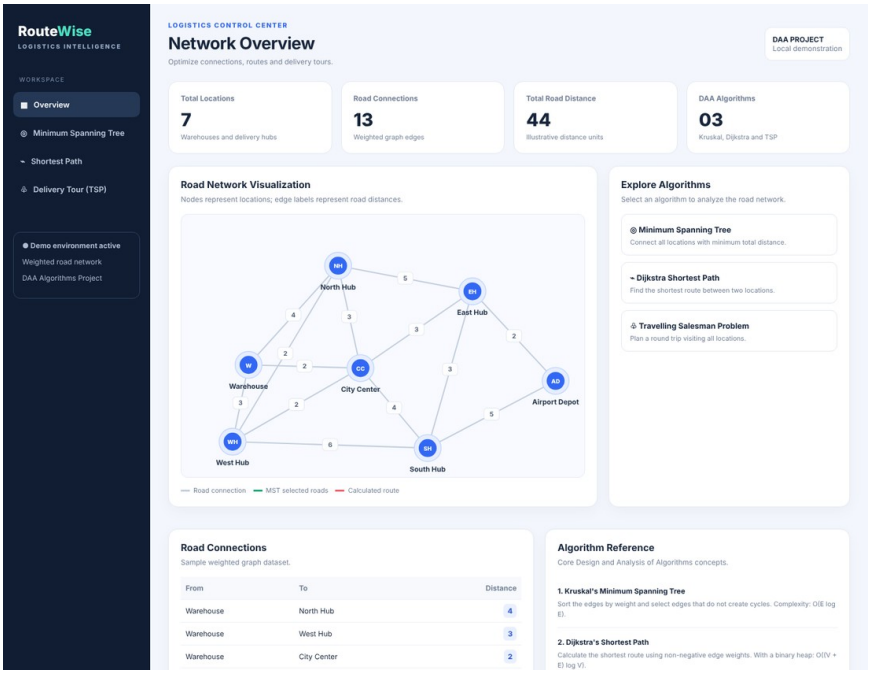


Figure A.2: Minimum Spanning Tree (Kruskal), 6 highlighted roads, total 14 units.

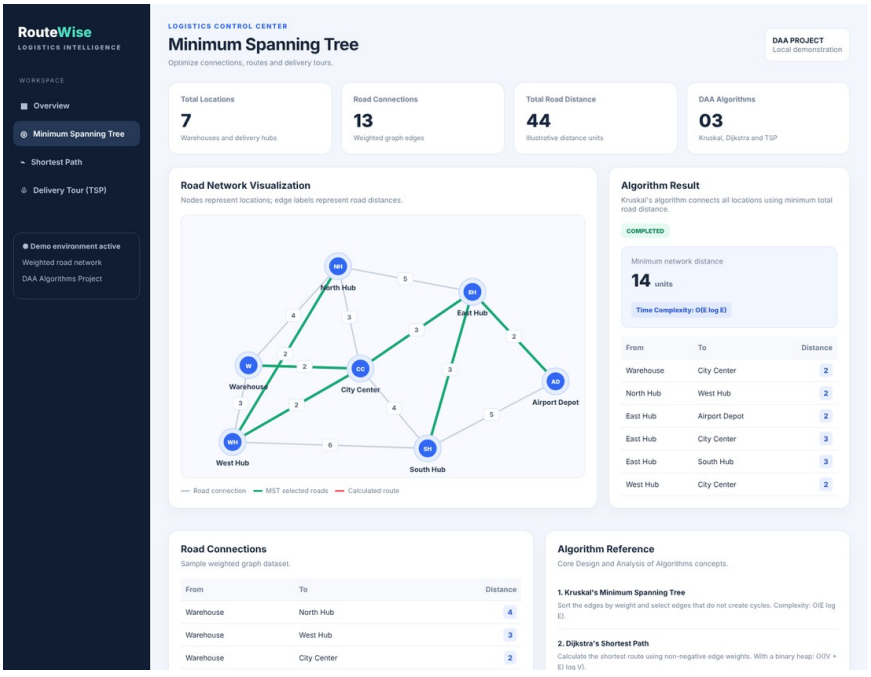


Figure A.3: Shortest path from Warehouse to Airport Depot (Dijkstra), 7 units.

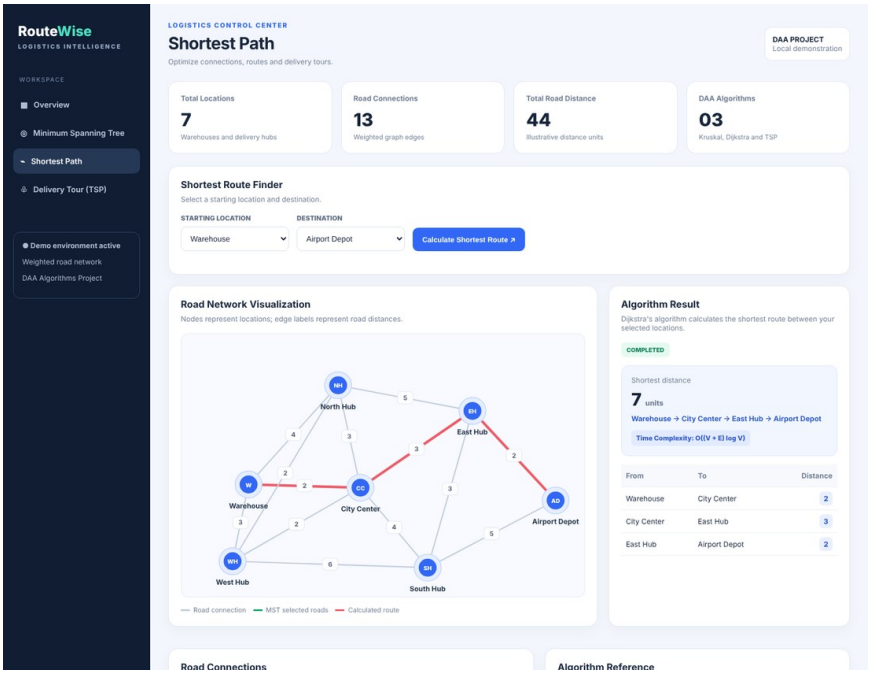


Figure A.4: Delivery tour (TSP) from Warehouse, best tour distance 23 units.

